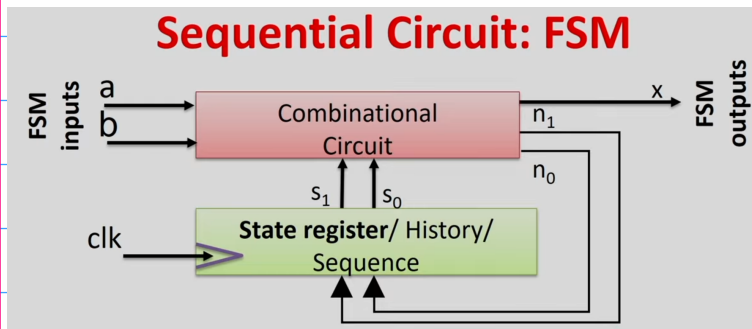


Notes - FSM - Sequential Circuit



2 types of FSM

Moore FSM: output is only function of state

Mealy FSM: output is function of state and inputs

Set Theoretic Description

Moore Machine is an ordered quintuple

$$M o o r e = (S, I, O, \delta, \lambda)$$

where

$$S = \{s_1, s_2, \dots, s_n\}$$

$$I = \{i_1, i_2, \dots, i_m\}$$

$$O = \{o_1, o_2, \dots, o_l\}$$

$$\delta = S \times I \rightarrow S$$

$$\lambda = S \rightarrow O$$

S = finite set of state ($\neq \emptyset$)

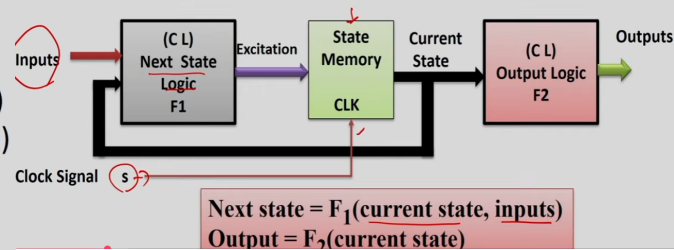
I = finite set of inputs ($\neq \emptyset$)

O = finite set of output ($\neq \emptyset$)

δ = next state function

λ = output function

Moore Machine: Moore FSM



Set Theoretic Description

Mealy Machine is an ordered quintuple

$$M e a l y = (S, I, O, \delta, \beta)$$

where

$$S = \{s_1, s_2, \dots, s_n\}$$

$$I = \{i_1, i_2, \dots, i_m\}$$

$$O = \{o_1, o_2, \dots, o_l\}$$

$$\delta = S \times I \rightarrow S$$

$$\beta = S \times I \rightarrow O$$

S = finite set of state ($\neq \emptyset$)

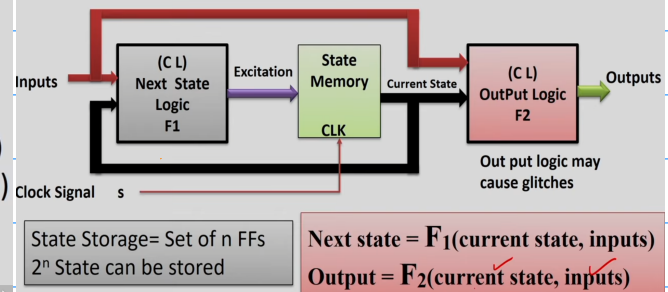
I = finite set of inputs ($\neq \emptyset$)

O = finite set of output ($\neq \emptyset$)

δ = next state function

λ = output function

Mealy Machine: Mealy FSM



Clock Synchronous FSM

All circuit used clock at same time

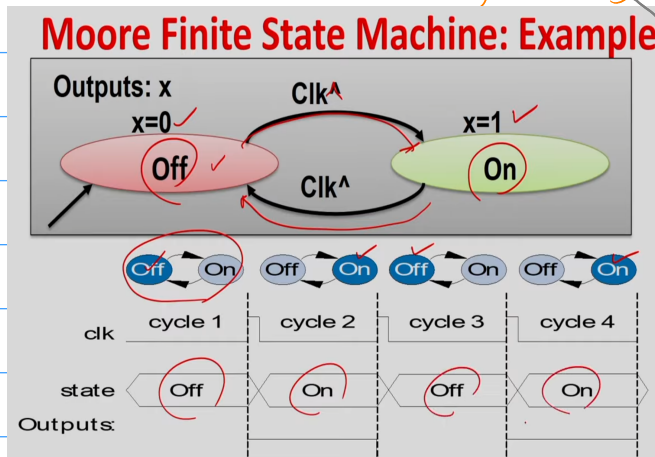
Sequential Circuit Design Process

- 1) Describe the problem in HLL like english
- 2) Model the problem using FSM
- 3) Represent the FSM in Tabular form
- 4) Derive number of FF require If input are 2^{K+1} then required $K+1$ FF
- 5) State Encoding (G.A. Optimal Encoding Optimization)
- 6) Derivation of - Next State : From Present State and Input
Output : From Present State (for mealy FSM : included input)
- 7) Derive combination function for NS logic, OP logic
- 8) Realize FSM by connecting the NS, OP logic, State Reg and Clock

Eg: Design Circuit changes output every clock cycle.

$x (0 \rightarrow 1) (1 \rightarrow 0)$
 2 states "off" ($x=0$) and "on" ($x=1$)

Transition on rising clock edge



Step 3 - Tabular Form

	Input		Output
	Clk	PS	$X = NS$
RE 1	0	1	1
RE 1	1	0	0

Step 4

2 States : 1 FF

Step 5: State Encoding

State 0: 0 State 1: 1

Step 6: Derivation of

Next State: From Present State & Input

Output: From Present State

Step 8 Realize FSM by NS, OP logic

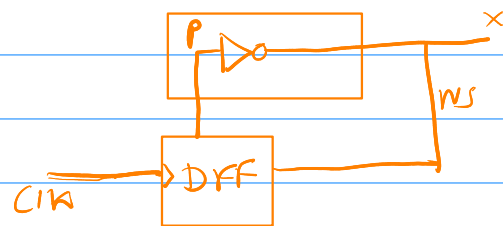
State Reg and Clock

Step 7 - Derive combinational logic for

NS logic & OP logic

$NS = \text{not } P$ $X = \text{not } P$

$P \rightarrow \neg P \rightarrow X_{NS}$



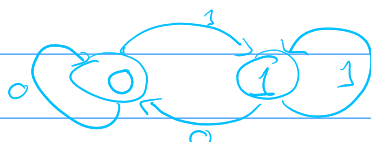
Eg 2 FSM for D-FF

Step 1: Next State is depend on data Input

Step 2 2 States

OFF ($S=0$) On ($S=1$) Transition on rising edge / No starting Point

Step 3 Model FSM



Step 6: Tabular Form

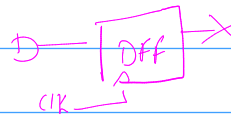
PS (Q)	I	NS (Q+1)
0	0	0
0	1	1
1	0	0
1	1	1

Step 4 2 State = 1 FF

Step 5 State encoding

State 0: 0 State 1: 1

Step 7 - $X=D$, $NS=D$ Stop: 8 Realize FSM

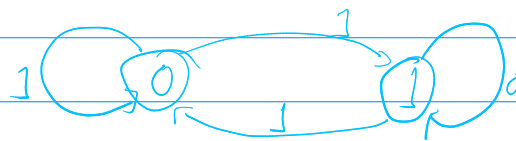


Eg-3 FSM for TFF

Step 1 - When input is toggle (Rising edge trigger $0 \rightarrow 1$, $1 \rightarrow 0$)

Step 2: On: $s=1$ 'Off' $s=0$

Step 3: Model and Table



Step 4 2 State = 1 FF

Step 5 State Encoding

0: 0 1: 1

Step 6

PS(Q)	I	NS(Q+t)
0	0	0
0	1	1
1	1	0
1	0	1

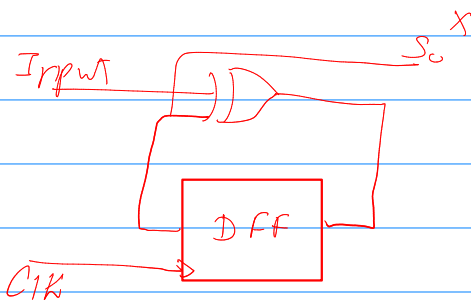
Step-7

$$X = Q$$

$$NS = Q'T + Q'T$$

$$= Q \oplus T$$

Step 8 Realise FSM



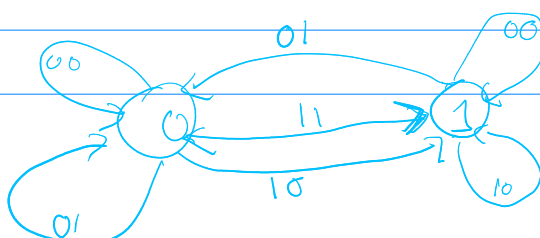
Eg JK flip flop using FSM

Step 1: JK FlipFlop where Output reset when $K=1, J=0$, toggle when $JK=1,1$
set to 1 when $J=1, K=0$ and ~~set to 0~~ ^{No change} when $J=0, K=0$

Step 2: 2 State on: $s=1$ 'Off' $s=0$

Rising edge clock trigger

Step 3 Model



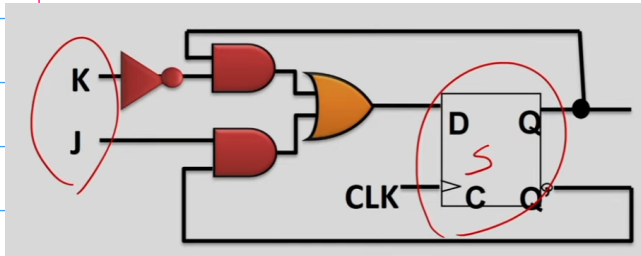
Step 6

PS(Q)	JK	NS(Q+t)
0	00	0
0	01	0
0	10	1
0	11	1
1	00	1
1	10	0
1	11	0
1	01	0

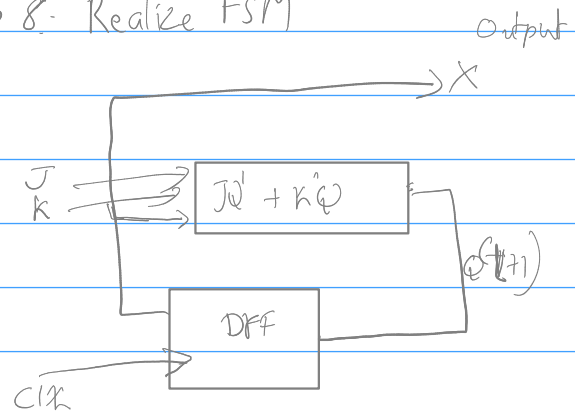
Step 7

$$NS = JQ' + K'Q$$

$$X = Q$$



Step 8: Realize FSM



Comparison : Mealy FSM Vs Moore FSM

- **Mealy** machines have **less states**
 - Outputs are on transitions (SxI) rather than states (n)
- **Moore** machines are **safer** to use
 - Outputs change at clock edge (always one cycle later)
- Mealy machines: input change can cause output change as soon as logic is done ∞
 - A **big problem** when two machines are interconnected
 - Asynchronous feedback **may** occur if one isn't careful

Comparison : Mealy FSM Vs Moore FSM

- **Mealy** machines **react faster** to inputs
 - react in same cycle – don't need to wait for clock
 - outputs **may** be considerably shorter than the clock cycle
 - in Moore machines, more logic **may** be necessary to decode state into outputs – there **may** be more gate delays after clock edge

FSM: State Minimization

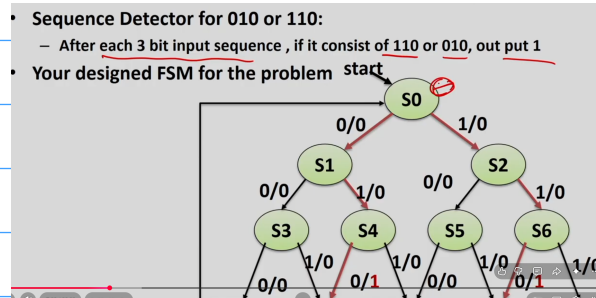
→ Reducing state will reduce state reg

→ State Equivalence : S1 and S2 are equivalent if for every input sequence applied to machine goes to same NS and Output

iff $S_1(t+1) = S_2(t+1)$ and $Z_1 = Z_2$ then $S_1 = S_2$

→ Distinguishable State : Two states S1 and S2 are distinguishable only if there exist at least one finite input sequence which produce different Outputs from S1 and S2

Method 1 : Row Matching
Eg Sequence Detector

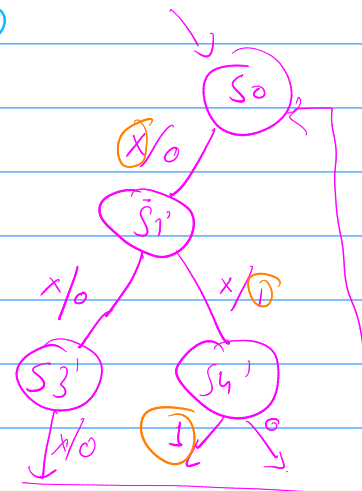


Tabular form

Input	PS	NS		Output	
		X=0	X=1	X=0	X=1
Reset	S ₀	S ₁	S ₂	0	0
0	S ₁	S ₃	S ₄	0	0
1	S ₂	S ₅	S ₆	0	0
00	S ₃	S ₀	S ₀	0	0
01	S ₄	S ₀	S ₀	1	0
10	S ₅	S ₀	S ₀	0	0
11	S ₆	S ₀	S ₀	1	0

(S₀)(S₁')(S₃')(S₄')

Reset	S ₀	S ₁ '	S ₁ '	0	0
0/1	S ₁ '	S ₃ '	S ₄ '	0	0
00/10	S ₃ '	S ₀	S ₀	0	0
01/11	S ₄ '	S ₀	S ₀	1	0



State Minimization : Partitioning

→ Form an initial partition (P₁) : that includes all the states.

→ Form an 2nd Partition (P₂)

- By separating the states into blocks corresponding to the next state value.

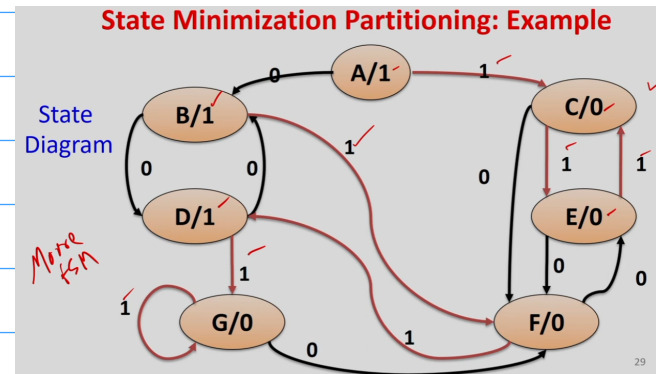
- Continue partitioning until

- Two successive partitions are the same (i.e. P_{N-1} = P_N)

- All state in any one block are equivalent

- Equivalent states can be combined into single state

PS	NS		Output
	$w=0$	$w=1$	
A	B	C	1
B	D	F	1
C	F	E	0
D	B	G	1
E	F	C	0
F	E	D	0
G	F	G	0



$$P_1 = \{ABD, CEF, G\}$$

$$P_2 = \{ABD, F, CEG\}$$

$$P_3 = \{AD, B, F, CEG\} \quad P_3 = P_4$$

1

In Detail Eg2. Partitioning

All Excitation table

$\bar{J}K$

Q	Q'	$\bar{J}$	K
0	1	0	x
0	1	1	x
1	0	x	1
1	1	x	0

D-FF

Q	Q'	D
0	0	0
0	1	1
1	0	0
1	1	1

T-FF

Q	Q'	T
0	0	0
0	1	1
1	0	1
1	1	0

Your transition table is:

State	X=0	X=1	Output
A	B	A	0
B	B	C	0
C	D	B	0
D	E	C	1
E	B	C	0

Only D has output 1, so D is the only final/accepting state.

Step 1: Initial partition $\{P_0\}$

Separate states based on output:

$\boxed{P_0 = \{D\}, \{A, B, C, E\}}$

So we have two groups:

Group 1 = $\{D\}$ → output 1
Group 2 = $\{A, B, C, E\}$ → output 0
Step 2: Check transitions

For each state, look at where 0 and 1 take us in terms of groups.

State A

A:

0 → B → $\{A, B, C, E\}$
1 → A → $\{A, B, C, E\}$

So:

$A \rightarrow \{A, B, C, E\}$
State B

B:

0 → B → $\{A, B, C, E\}$
1 → C → $\{A, B, C, E\}$

So:

$B \rightarrow \{A, B, C, E\}$
State C

C:

0 → D → $\{D\}$
1 → B → $\{A, B, C, E\}$

So:

$C \rightarrow \{D\}$
State D

D:

0 → E → $\{A, B, C, E\}$
1 → C → $\{A, B, C, E\}$

So:

$D \rightarrow \{D\}$

But D is already separated because its output is different, so that's fine.

State E

E:

0 → B → $\{A, B, C, E\}$
1 → C → $\{A, B, C, E\}$

So:

$E \rightarrow \{A, B, C, E\}$
Step 3: Split $\{A, B, C, E\}$

We have:

State	X=0 group	X=1 group	Signature
A	G ₂	G ₃	(G_2, G_3)
B	G ₂	G ₃	(G_2, G_3)
C	G ₁	G ₃	(G_1, G_3)
E	G ₂	G ₃	(G_2, G_3)

Therefore C must be separated.

So:

$\boxed{P_1 = \{D\}, \{C\}, \{A, B, E\}}$
Step 4: Repeat

Now use the new groups:

$G_1 = \{D\}$ $G_2 = \{C\}$ $G_3 = \{A, B, E\}$

Check $\{A, B, E\}$.

A

0 → B → G₃

1 → A → G₃

$A \rightarrow G_3$

B

0 → B → G₃

1 → C → G₂

$B \rightarrow (G_3, G_2)$

E

0 → B → G₃

1 → C → G₂

$E \rightarrow (G_3, G_2)$

Therefore:

$A \neq (B, E)$

So split:

$\{A, B, E\} \rightarrow \{A\}, \{B, E\}$

Therefore:

$\boxed{P_2 = \{D\}, \{C\}, \{A\}, \{B, E\}}$
Step 5: Check $\{B, E\}$

Now compare B and E.

B

0 → B → $\{B, E\}$

1 → C → $\{C\}$

E

0 → B → $\{B, E\}$

1 → C → $\{C\}$

Both have exactly the same partition behavior:

$B = (B, E), \{C\}$ $E = (B, E), \{C\}$

So B and E remain together.

No more splitting is possible.

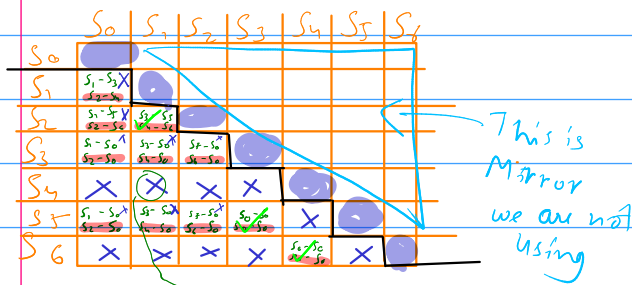
Therefore final partition is:

$\boxed{P_f = \{A\}, \{B, E\}, \{C\}, \{D\}}$
Final minimized DFA

We can merge:

B=E

Implication chart Method



Check for Input 1 NS
S2-S4 checkbox

S4/S4 Output is 1

S3 & S5 are equivalent
S4 & S6 are equivalent
S1 & S2 are equivalent

Input	PS	NS		Output	
		X=0	X=1	X=0	X=1
Reset	S0	S1'	S1'	0	0
0 or 1	S1'	S3'	S4'	0	0
00 or 10	S3'	S0	S0	0	0
01 or 11	S4'	S0	S0	1	0

FSM : Tabular Form

Input	PS	NS		OUTPUT	
		X=0	X=1	X=0	X=1
Reset	S0	S1	S2	0	0
0	S1	S3	S4	0	0
1	S2	S5	S6	0	0
00	S3	S0	S0	0	0
01	S4	S0	S0	1	0
10	S5	S0	S0	0	0
11	S6	S0	S0	1	0

Implication Chart Method

Filling in the Implication Chart

- Entry X_{ij} — Row is S_i , Column is S_j
- S_i is equivalent to S_j if outputs are the same and next states are equivalent
- X_{ij} contains the next states of S_i, S_j which must be equivalent if S_i and S_j are equivalent
- If S_i, S_j have different output behavior, then X_{ij} is crossed out

#

State Encoding

State Assignment - Assign state to Unique binary number

State Encoding

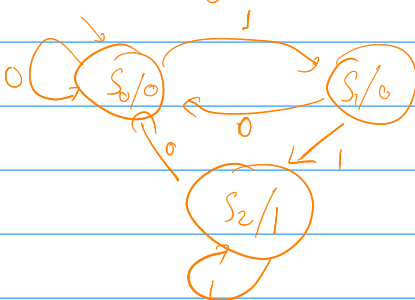
- The cost & delay of FSM implementation depends on encoding of symbolic states.
 - e.g., 4 states can be encoded in $4! = 24$ different ways
- There are more than $n!$ different encodings for n states.
 - Exploration of all encodings is impossible, therefore heuristics are used
- Heuristics Used
 - One-hot encoding, Minimum-bit change, Prioritized adjacency

State Assignment Problem

- Some state assignments are better than others.
- The state assignment influences the complexity of the state machine.
 - The combinational logic required in the state machine design is dependent on the state assignment.
- Types of state assignment
 - Binary encoding: 2^N states $\rightarrow$ N Flip-Flops
 - Gray-code encoding: 2^N states $\rightarrow$ N Flip-Flops
 - One-hot encoding: N states $\rightarrow$ N Flip-Flops

Eg

State Diagram



FSM: State Assignment

Present State	Next State		Output
	w = 0	w = 1	
S ₀	S ₀	S ₁	0
S ₁	S ₀	S ₂	0
S ₂	S ₀	S ₂	1

Present State			Next State				Output
			w = 0 ✓		w = 1 ✓		
	Q_A	Q_B	Q_A^+	Q_B^+	Q_A^+	Q_B^+	z
S_0 ✓	0	0	0	0	1	1	0
S_1 ✓	0	1	0	0	1	0	0
S_2 ✓	1	0	0	0	1	0	1
S_3 ✓	1	1	d	d	d	d	d

Using Binary Encoding
for the State Assignment

$$D_B = wQ_A'Q_B'$$
$$D_A = w(Q_A + Q_B)$$
$$Z = Q_A$$

Using Binary Encoding for the State Assignment

$$\begin{aligned} D_B &= wQ_A'Q_B' \\ D_A &= w(Q_A + Q_B) \\ Z &= Q_A \end{aligned}$$

Present State			Next State				Output
			w = 0		w = 1		
	Q_A	Q_B	Q_A^+	Q_B^+	Q_A^+	Q_B^+	z
S_0	0	0	0	0	1	1	0
S_1	0	1	0	0	1	1	0
S_2	1	1	0	0	1	1	1
	1	0	d	d	d	d	d

Using Gray-code Encoding
for the State Assignment

$D_A = w \cdot Q_B$
 $D_B = W$
 $Z = O.$

44

Using Gray-code Encoding for the State Assignment

$$\begin{aligned} D_A &= w.Q_B \\ D_B &= W \\ Z &= Q_A \end{aligned}$$

Present State		Next State			
		w = 0		w = 1	
	Q_A Q_B Q_C	Q_A^+ Q_B^+ Q_C^+	Q_A^+ Q_B^+ Q_C^+	Q_A^+ Q_B^+ Q_C^+	Q_A^+ Q_B^+ Q_C^+
S_0	0 0 1	0 0 1	0 0 1	0 1 0	0 1 0
S_1	0 1 0	0 0 1	0 0 1	1 0 0	1 0 0
S_2	1 0 0	0 0 1	0 0 1	1 0 0	1 0 0

Using One-hot Encoding for the State Assignment

For each state only one flip-flop is set to 1.

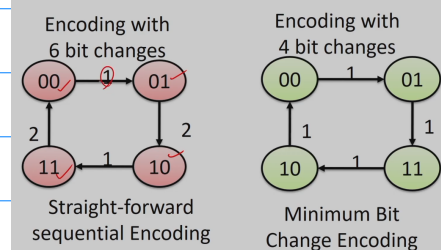
Remaining combination of state variables are not used

$$\begin{aligned} D_A &= w.Q_C' \\ D_B &= w.Q_C \\ D_C &= w' \end{aligned}$$

Minimum-bit Change Encoding

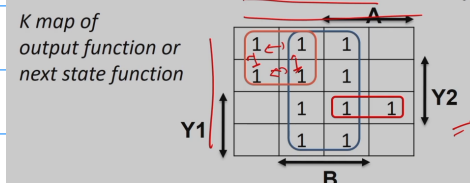
- Assigns codes to states so that
 - The total number of **bit changes** for all state transitions is **minimized**.
- In other words, if every arc in the state diagram has a **weight**
 - That is equal to the **number of bits** by which the **source** and **destination** encoding **differ**
 - This strategy would select the one that **minimizes the sum** of all these weights.

Minimum-bit Change Encoding



Idea of Adjacency of KMAP

- Inputs are A and B
- State variables are $Y1$ and $Y2$
- An output is $F(A, B, Y1, Y2)$ or $F(Y1, Y2)$
- A next state function is $G(A, B, Y1, Y2)$
- Larger clusters produce smaller logic function.
- Clustered minterms differ in one variable.



Things to take Care

Size of an Implementation

- Number of **product terms** determines
 - Number of **gates**
- Number of **literals** in a product term determines
 - Number of gate **inputs**, which is proportional to number of transistors.
- Hardware area proportional to
 - Total number of literals
- Examples of four minterm functions:
 - $F1 = ABCD + A'B'C'D' + A'B'CD + A'B'CD$ has **16 literals**
 - $F2 = ABC + A'C'D$ has **6 literals**

3 Rules

^{Priority} #1) Common Dest: States that have the same next state for some fixed input should be assigned logically adjacent codes.

#2) Common Source: States that are the next states of the same state under logically adjacent inputs, should be assigned logically adjacent codes.

#3) Common Output: States that the same output values for the same input value, should be assigned logically adjacent codes.